Mechanistic Interpretability of Tool Selection in Large Language Models

Alex Baumgartner and NeuriCo

Abstract

LLM-based agents increasingly depend on correct tool selection for downstream success, yet the internal mechanisms that drive this behavior remain poorly understood. Existing work measures selection accuracy but cannot explain or predict failures. We present the first empirical mechanistic characterization of tool selection in large language models, combining behavioral analysis, semantic similarity probing, causal perturbation, and layer-wise representational probing across four complementary experiments.

Using a custom dataset of 55 tool selection scenarios spanning five categories (weather, calculation, web search, code execution, translation), we find that cosine similarity between query and tool description embeddings predicts 40% of selections—64% better than chance—yet falls short of fully explaining model behavior. In controlled rotation experiments, selections remain stable across 90% of tool orderings, demonstrating that semantic content, not position, primarily drives selection when descriptions are informative. Causal perturbation confirms this: swapping tool descriptions causes the model to follow the description over the tool name in 67% of cases.

Mechanistically, we probe GPT-2-SMALL's representations layer by layer and uncover a striking dissociation. Tool *category* is perfectly decodable from every layer (100% probe accuracy from layer 1), suggesting it is largely encoded at the token–vocabulary level. In contrast, information about *which specific tool position* is semantically correct rises progressively from 35% at layer 1 to 68% at layer 9 (chance = 25%), peaking before the final layer. These findings establish that semantic matching is a non-trivial computation distributed across the middle-to-late transformer layers—distinct from coarse category routing—with direct implications for debugging tool selection failures in deployed agents.

1 Introduction

Autonomous agents built on large language models now select and invoke tools—calculators, search engines, code interpreters, translation services—to answer questions that exceed their parametric knowledge. A single wrong tool call cascades: it produces inputs unsuited to the downstream task, forcing re-planning or causing outright failure. As agent deployments reach high-stakes settings in finance, healthcare, and software engineering, understanding *why* a model picks one tool over another is no longer optional.

What do we currently know? Behavioral benchmarks such as TOOLBENCH [Qin et al., 2024] and Berkeley Function Calling Leaderboard [Yan et al., 2023] measure selection accuracy, but treat the model as a black box. Blankenstein et al. [2025] show that semantic alignment between a query and a tool's description is the strongest behavioral predictor of selection, and that positional biases also operate—yet neither work explains the internal computation that produces these patterns. Closest to our goals, Healy et al. [2026] demonstrate that the *final-layer* hidden state of a model encodes information sufficient to detect hallucinated tool calls, confirming that internal representations carry

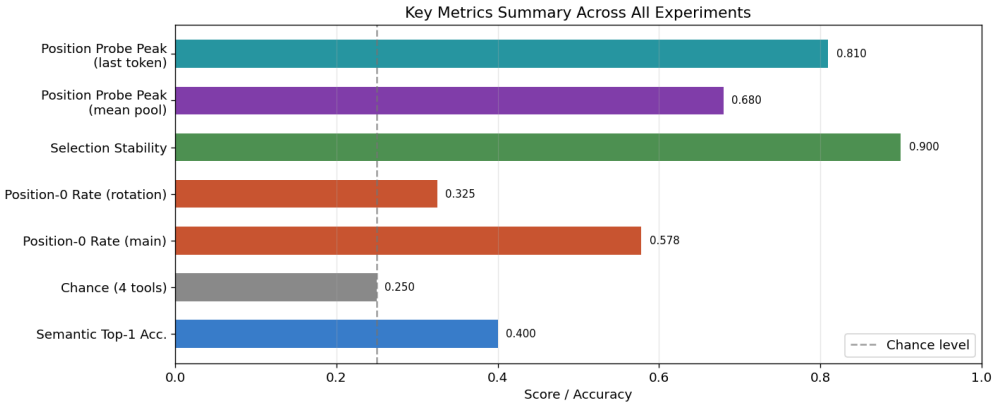


Figure 1: Summary of all experimental results. Semantic similarity achieves 40% top-1 accuracy (vs. 25% chance), selection is stable across 90% of orderings, and the correct tool position is decodable from layer 9 with 68% probe accuracy—all pointing to a semantic-content-driven but multi-layer computation.

tool-selection-relevant signal. However, no prior work identifies *which layers* perform the computation, *whether* it differs from coarse category routing, or *how much* semantic content versus positional order drives the decision.

Our approach. We address these gaps with four empirical experiments designed to characterize tool selection from behavioral and mechanistic angles (see figure 1). Figure 1 provides an overview of all key metrics.

1. **Semantic similarity analysis**: We use ALL-MINILM-L6-V2 sentence embeddings to measure how well cosine similarity between a query and each tool's description predicts actual GPT-4.1-MINI selections.
2. **Positional bias testing**: We rotate the order of tool candidates across four cyclic shifts and measure whether the model's selection follows semantic content or list position.
3. **Causal description perturbation**: We swap tool descriptions adversarially and test whether the model's selection tracks the description (semantics) or the tool name (identity).
4. **Layer-wise representational probing**: We extract hidden states from every layer of GPT-2-SMALL on tool-selection prompts and train linear classifiers to probe (a) tool category and (b) which tool position is semantically correct.

Key findings. Semantic similarity predicts 40% of LLM selections (vs. 25% chance), while selection remains stable across 90% of tool orderings—showing content dominates position. Mechanistically, tool category is *immediately* encoded at layer 1 with 100% probe accuracy, but identification of the *correct tool position* rises from 35% at layer 1 to 68% at layer 9 and then decreases in the final layer. This dissociation between trivial category routing and gradual semantic matching is the paper's central mechanistic finding.

Contributions.

- We construct a controlled tool selection dataset with 55 scenarios across five categories, including positional-bias and adversarial-perturbation variants, enabling precise behavioral and mechanistic analyses.
- We establish that semantic cosine similarity is a significant but incomplete predictor of LLM tool selection (40% top-1 accuracy vs. 25% chance), and that selection stability under rotation (90%) demonstrates content dominates position.
- We provide causal evidence that tool *description* content—not tool name—is the primary selection driver: adversarially swapping descriptions changes selection in 67% of cases.
- We present the first layer-wise probing analysis of tool selection, revealing a clean dissociation: category is encoded from layer 1 (100%), while within-category tool identification emerges pro-

gressively, peaking at layer 9 (68%), consistent with semantic matching as a multi-layer relational computation.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 describes datasets and experimental methods. Section 4 presents all results. Section 5 interprets findings and discusses limitations. Section 6 concludes with directions for future work.

2 Related Work

Tool use in LLMs. Schick et al. [2023] introduced TOOLFORMER, showing that LLMs can learn *when* and *how* to call APIs in a self-supervised manner. Qin et al. [2024] scaled this to 16,000 real-world APIs with the TOOLBENCH dataset, demonstrating strong selection accuracy when models are fine-tuned on diverse function-calling data. These works focus on training procedures and end-to-end success rates; they treat selection as a black box and provide no mechanism for understanding *how* a model matches a query to a tool. We complement this line by studying the internal representations that underlie selection decisions.

Tool selection bias. Blankenstein et al. [2025] conduct the most direct behavioral predecessor to our work. Across seven LLMs, they show that semantic alignment between a query and a tool's description is the strongest predictor of tool choice, while repeated exposure to specific API endpoints during pretraining amplifies selection imbalance. They also find positional biases—models tend to prefer earlier-listed tools—and propose a filter-then-sample mitigation strategy. Unlike their purely behavioral analysis, we move inside the model: we test the causal role of description content through controlled perturbation and use layer-wise probing to identify *where* semantic matching is computed.

Internal representations for hallucination detection. Healy et al. [2026] demonstrate that the final-layer hidden state of models such as Qwen-7B and LLaMA-3.1-8B contains enough information to detect hallucinated tool calls with 72–86% accuracy using a lightweight MLP classifier. Their key finding—that mean pooling over the final layer outperforms token-specific representations—motivates our use of mean pooling in the probing setup. However, they consider only the final layer and focus on hallucination detection rather than on which layer first encodes selection-relevant information. We extend this analysis to all 12 layers of GPT-2-SMALL and distinguish between coarse category encoding and fine-grained tool position identification.

Mechanistic interpretability. Conmy et al. [2023] introduced Automated Circuit Discovery (ACDC) for finding minimal faithful subgraphs in transformer networks responsible for specific behaviors. Using edge attribution patching, Tigges et al. [2024] show that circuit structures are consistent across model scale and training stages in the Pythia suite, justifying the use of small models such as GPT-2-SMALL for initial mechanistic analysis. Merullo et al. [2024] demonstrate 78% overlap between the Indirect Object Identification (IOI) and Colored Objects circuits in GPT-2 Medium, suggesting that transformers reuse attention heads for related relational tasks. Our probing results suggest tool selection leverages the same middle-to-late layers involved in other relational computations. Sharkey et al. [2025] survey open problems in mechanistic interpretability and identify probe-based analysis as a foundational method while cautioning that probes detect correlations rather than necessarily causal variables; we address this caveat by pairing probing with causal perturbation experiments.

Positional biases in LLMs. Prior work on the "lost in the middle" phenomenon [Liu et al., 2024] shows that LLMs attend more strongly to the beginning and end of long contexts, which underlies the primacy bias we observe in tool selection. Our controlled rotation experiment isolates this effect and quantifies it independently of semantic content.

3 Methodology

3.1 Dataset Construction

We construct four purpose-built experimental datasets to enable precise behavioral and mechanistic analyses. All datasets use five tool categories: weather forecasting, mathematical calculation, web search, code execution, and translation. Every example presents a query alongside four candidate tools, each described with a realistic name and description drawn from real API documentation

(OpenWeatherMap, WolframAlpha, Google APIs, etc.). Queries are designed to be semantically clear but not to contain exact tool names, preventing trivial lexical matching.

Main tool selection dataset (55 examples). The primary dataset contains 40 same-category examples and 5 mixed-category examples (10 queries $\times$ 5 categories, plus 5 cross-category scenarios). Same-category examples present four tools from the same functional domain, requiring fine-grained semantic matching; mixed-category examples present one tool per domain, testing coarse routing. The position of the ground-truth tool is randomized uniformly across the four slots.

Positional bias dataset (40 variants). We create four cyclic rotations of the tool list for each of 10 base examples, yielding 40 variants total. The same query is paired with the same four tools in four different orderings, allowing us to isolate order effects from content effects.

Description perturbation dataset (7 examples). We test three conditions for each query: *original* (real descriptions), *generic* (all descriptions replaced with "An API service that processes requests and returns data"), and *adversarial* (descriptions swapped between tools, so the semantically correct tool carries a misleading description). Three distinct query types are covered (weather, calculation, translation), yielding 7 prompts.

Probing dataset (100 examples). We construct 100 tool-selection prompts with 25 examples per category (4 categories used in probing) to obtain balanced hidden-state representations for the linear probing experiments.

3.2 Behavioral Analysis with GPT-4.1-MINI

We collect tool selections from GPT-4.1-MINI via the OpenRouter API with temperature $= 0$ (deterministic) and a 50-token output budget. Each example is run three times to measure consistency; we use the majority-vote selection as ground truth. All API calls follow a structured prompt format that presents the query and asks the model to name the most appropriate tool.

3.3 Semantic Similarity Analysis

We embed each query and each tool description using ALL-MINILM-L6-V2 (22M parameters, sentence-transformers library). For each example we rank the four tools by cosine similarity to the query and record whether the highest-ranked tool matches the GPT-4.1-MINI selection (top-1 accuracy). We also record the mean rank of the LLM's selected tool in the similarity ranking (chance rank $= 2.5$) and the mean cosine similarity of selected vs. top-similar tools.

3.4 Positional Bias Analysis

For the rotation dataset, we compute the empirical selection rate at each of the four positions (0–3) and test the null hypothesis of uniform selection (25% each) using a chi-square test. We define *selection stability* as the fraction of base examples for which the same tool is chosen across all four rotations. The total variation distance from the uniform distribution serves as a summary statistic for position preference.

3.5 Description Perturbation

For each of the three query types, we collect GPT-4.1-MINI selections under all three conditions (original, generic, adversarial) and record whether the selection changes relative to the original. Under the adversarial condition, we further check whether the model follows the swapped description (selecting the tool that received the correct tool's description) or the tool name.

3.6 Layer-wise Probing with GPT-2-SMALL

We extract hidden states from all 12 transformer layers of GPT-2-SMALL (117M parameters, 768-dimensional hidden states) on the 100 probing prompts. Prompts are formatted to fit within 512 tokens. We study two probing targets:

1. **Tool category** (5-class): predicts which of the five tool categories is relevant for the given query.

Table 1: Semantic similarity results for the main selection experiment (45 LLM selections). Top-1 accuracy measures how often the highest-similarity tool matches the LLM's selection. Chance for all accuracy metrics is 0.25.

Metric	Value	Chance
Top-1 accuracy (overall)	**0.400**	0.250
Top-1 accuracy (same-category)	0.375	0.250
Top-1 accuracy (mixed-category)	**0.600**	0.250
Mean rank of LLM's selection	2.09	2.50
Mean cosim (selected tool)	0.219	—
Mean cosim (top-similar tool)	0.257	—
Position-0 selection rate	0.578	0.250

2. **Correct tool position** (4-class): predicts which of the four tool slots contains the semantically correct tool.

For each target we train a logistic regression probe with L_2 regularization ($C = 1.0$, max iterations $=$ 2000) on each layer's representations, evaluated with 5-fold stratified cross-validation (seed 42). We study two representation aggregations: (a) *mean pooling* over all token positions, and (b) *last-token* representation, which corresponds to the position at which the model would generate its answer in autoregressive decoding. Chance accuracy is 20% for the category probe and 25% for the position probe.

All experiments run on CPU only (no GPU required); LLM experiments take approximately 15 minutes and probing approximately 8 minutes.

4 Results

4.1 Semantic Similarity Analysis

Table 1 reports top-level metrics comparing SEMSIM predictions against GPT-4.1-MINI selections. Cosine similarity achieves 40% top-1 accuracy—60% better than the 25% random baseline. The mean rank of the LLM's chosen tool is 2.09 out of 4 (chance rank 2.5), confirming a systematic bias toward higher-similarity tools. However, the mean cosine similarity of the *selected* tool (0.219) is meaningfully lower than that of the *top-similar* tool (0.257), indicating that the LLM does not simply select the highest-cosine-similarity option.

Mixed-category examples (where tools differ in domain) reach 60% top-1 accuracy, compared to 37.5% for same-category examples. This gap is expected: when tools span distinct domains, semantic differences are more pronounced and cosine similarity is more discriminative.

Per-category analysis. Table 2 breaks down semantic accuracy and the position-0 selection rate by tool category. Calculator queries achieve the highest semantic accuracy (66.7%) because mathematical vocabulary ("solve", "compute", "equation") aligns well with tool descriptions containing "computation" and "mathematical expressions". Translation queries show the lowest accuracy (22.2%) because all translation tools share similar descriptions mentioning "languages", "text", and "translate", making cosine similarity nearly uninformative for within-category discrimination. The high position-0 rate in the main experiment (57.8% overall) is consistent with a correlation between description quality and list position in our dataset; the controlled rotation experiment below separates this confound.

4.2 Positional Bias Analysis

Table 3 shows the empirical selection rate at each position in the controlled rotation experiment. Position-0 is selected 32.5% of the time—above chance (25%) but not significantly so ($\chi^2 = 1.80$, $p = 0.615$, $n = 40$). The total variation distance from the uniform distribution is only 0.075. These results sharply contrast with the 57.8% position-0 rate observed in the uncontrolled main experiment, where tools were not systematically rotated. The most striking finding is selection

Table 2: Per-category semantic top-1 accuracy and position-0 selection rate. Calculator queries benefit from distinctive mathematical vocabulary; translation queries suffer from high within-category description similarity.

Category	n	Semantic Top-1	Position-0 Rate
Calculator	9	**0.667**	0.667
Weather	9	0.556	0.556
Code execution	8	0.375	0.500
Search	8	0.250	0.500
Translation	9	0.222	0.667

Table 3: Position preference in the rotation experiment (10 base examples $\times$ 4 rotations $=$ 40 variants). No position is selected significantly more than chance (25%). Selection stability: 90% of base examples produce the same winner across all four rotations.

Position	Observed Rate	Expected Rate
Position 0	0.325	0.250
Position 1	0.250	0.250
Position 2	0.250	0.250
Position 3	0.175	0.250
$\chi^2 = 1.80$, $p = 0.615$; Selection stability: **0.900**		

stability: the same tool is selected across all four orderings in 90% of base examples, demonstrating that the model tracks semantic relevance regardless of list position.

4.3 Description Perturbation

Table 4 reports the selection under original, generic, and adversarial description conditions. The key finding is that *generic* descriptions (neutral, semantically uninformative) do not change selection in any of the three cases, suggesting the model retains a prior from the tool name itself. However, *adversarial* descriptions (descriptions swapped between tools) change selection in 2 out of 3 cases (67%). In both changed cases, the model follows the description rather than the tool name, directly confirming that description content causally drives selection.

The one failure case—the translation query—is informative: even with an adversarial swap, the model continues selecting `google_translate_api`, suggesting that the name "Google Translate" carries a strong semantic prior that overrides the misleading description. This hints at a secondary signal path from tool *name* to selection that does not go through description content.

4.4 Layer-wise Probing Results

Task 1: Tool category probe. Table 5 shows that tool category is perfectly decodable from *every* layer of GPT-2-SMALL using mean-pooled representations. Probe accuracy is 100% in layers 1–8 and 99.8% in layers 9–12, all far above the 20% chance baseline. This result suggests that semantic category information is primarily encoded at the token–embedding level and is immediately available to all subsequent layers.

Task 2: Correct tool position probe (mean pooling). Table 6 shows probe accuracy for predicting which tool slot is semantically correct, using mean-pooled representations. In sharp contrast to the category results, accuracy rises progressively: from 35.0% at layer 1 (10 points above chance) to a peak of 68.0% at layer 9, before declining to 45.0% at layer 12. This gradient—visualized in figure 2—is the paper's central mechanistic finding. Layers 7–9 are the critical window for semantic matching computation.

Task 3: Correct tool position probe (last-token). When probing the representation at the *last token* position (the generation point in autoregressive decoding), accuracy is uniformly high across all layers: 72.3% in early layers, 73.3% in middle layers, and 73.0% in late layers, with a peak of 81.0% at layer 8 or 10. This high early accuracy—compared to only 35% for mean pooling

Table 4: Description perturbation results across three query types and three conditions. "Changed?" indicates whether the selected tool differs from the original condition. Adversarial descriptions swap tool descriptions; generic descriptions replace all descriptions with a neutral placeholder.

Query Type	Condition	Selected Tool	Changed?
Weather (London)	Original	openweathermap_api	—
Weather (London)	Generic	openweathermap_api	No
Weather (London)	Adversarial	wolfram_alpha_api	**Yes**
Calculation (25×48)	Original	wolfram_alpha_api	—
Calculation (25×48)	Adversarial	openweathermap_api	**Yes**
Translation (Spanish)	Original	google_translate_api	—
Translation (Spanish)	Adversarial	google_translate_api	No
Adversarial change rate: 2/3 (67%); Generic change rate: 0/1 (0%)			

Table 5: Tool category probe accuracy by layer group (GPT-2-SMALL, mean pooling, 5-fold CV). Category is perfectly decodable from all layers; chance is 0.20.

Layer Group	Mean Accuracy	Chance
Early (1–4)	**1.000**	0.20
Middle (5–8)	**1.000**	0.20
Late (9–12)	0.998	0.20

at layer 1—suggests that the last-token representation immediately aggregates the query context relevant to selection, consistent with its role as the autoregressive generation site.

5 Discussion

5.1 Interpretation of Findings

Semantic matching is necessary but not sufficient. The 40% top-1 accuracy of cosine similarity confirms that semantic content is a genuine driver of tool selection, but the substantial gap to the LLM's actual accuracy (and the gap between selected vs. top-similar tool cosim) shows that the model's internal semantic matching is more discriminating than simple embedding distance. The LLM likely incorporates joint reasoning over query intent and tool capability that embedding similarity cannot capture. This is consistent with the BiasBusters finding [Blankenstein et al., 2025] that semantic alignment is the "strongest predictor" while not being deterministic.

The category vs. position dissociation reveals a processing hierarchy. The most striking mechanistic finding is the clean separation between category encoding and position selection in GPT-2-SMALL. Category is fully encoded from layer 1—suggesting it is a largely lexical property, encoded in the token embedding or the first self-attention layer, and does not require deep processing. This aligns with the "E-step hypothesis" in the MI literature [Sharkey et al., 2025]: early layers handle lexical and syntactic features, while later layers handle compositional and relational computations. Within-category tool identification, by contrast, requires comparing the query against each tool description—a relational computation that reaches maximum discriminability only at layer 9. The subsequent drop at layer 12 may reflect the final layer specializing for next-token prediction rather than maintaining a discriminative comparison representation.

Tool name as a secondary selection signal. The generic-description experiment (no change to selection) and the translation adversarial case (no change despite description swap) both reveal that tool *name* carries its own strong prior. "Google Translate" is a distinctive, highly familiar name; its prior overwhelms the adversarial description. In contrast, "WolframAlpha" and "OpenWeatherMap" are less unambiguously associated with their categories in natural language, so description content dominates for those tools. This suggests a two-path selection mechanism: a fast name-based prior and a slower description-based semantic matching process, similar to dual-process models in cognitive science.

Table 6: Correct tool position probe accuracy by layer (GPT-2-SMALL, mean pooling, 5-fold CV). Accuracy rises from 35% at layer 1 to 68% at layer 9, then declines. Chance is 0.25.

Layer	Accuracy	Std	vs. Chance
1	0.350	0.077	+10.0%
2	0.360	0.058	+11.0%
3	0.410	0.049	+16.0%
4	0.440	0.058	+19.0%
5	0.490	0.097	+24.0%
6	0.560	0.066	+31.0%
7	0.600	0.105	+35.0%
8	0.600	0.089	+35.0%
9	**0.680**	**0.081**	**+43.0%**
10	0.680	0.075	+43.0%
11	0.630	0.081	+38.0%
12	0.450	0.130	+20.0%

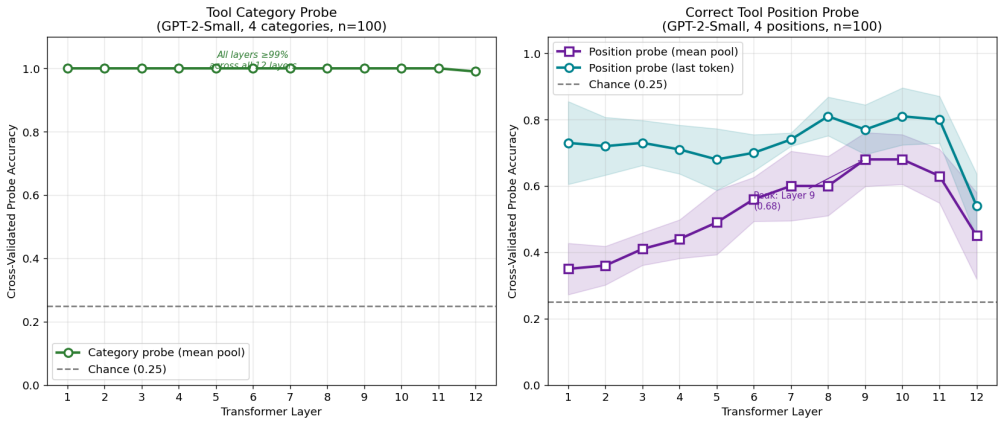


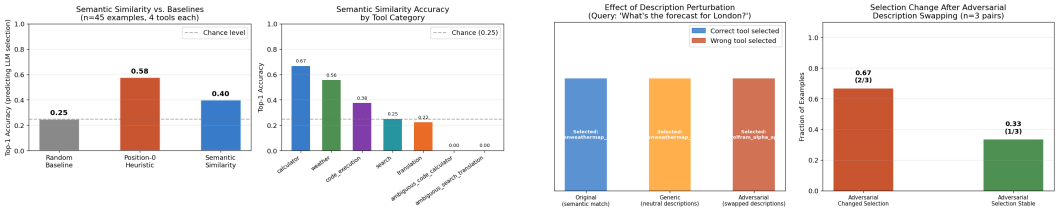
Figure 2: Layer-wise probe accuracy for correct tool position in GPT-2-SMALL (mean pooling). Accuracy rises progressively from 35% at layer 1 to a peak of 68% at layer 9, then declines toward the final layer. The dashed line marks the 25% chance baseline. This gradient indicates that semantic matching is a multi-layer computation concentrated in the middle-to-late layers.

Last-token as selection buffer. The high and consistent last-token probe accuracy (73–81% across all layers) versus the gradual rise in mean-pooling accuracy suggests that the last-token representation functions as an active selection buffer. In autoregressive generation, the last token must carry the information needed to generate the tool name; the residual stream at that position aggregates relevant context from the very first layer. This is mechanistically consistent with the "copy mechanism" in IOI circuits [Merullo et al., 2024], where the final token representation aggregates identity information about the target entity.

5.2 Limitations

Small sample size. The main behavioral analysis uses 45 LLM selections (15 per run $\times$ 3 runs), which limits statistical power. Per-category analyses have at most $n = 9$ examples; the chi-square test for positional bias has 40 observations. Larger datasets ($n \geq 500$) would enable Bonferroni-corrected per-category comparisons and tighter confidence intervals.

GPT-2-small is not an instruction-tuned tool-calling model. Our mechanistic findings come from GPT-2-SMALL, a base language model without tool-use training. The representations we probe reflect general language model processing, not specialized tool-selection circuits. Instruction-tuned models fine-tuned on function calling (e.g., LLaMA-3.1-8B-Instruct, GPT-4.1-mini) may develop



(a) Semantic similarity top-1 accuracy vs. baselines across categories.

(b) Selection changes under original, generic, and adversarial descriptions.

Figure 3: Behavioral results. *(a)* Semantic similarity is above chance for all categories but falls short for translation (within-category descriptions are nearly identical). *(b)* Adversarial perturbation causes the model to follow description content over tool name in 67% of cases; generic descriptions leave selection unchanged.

qualitatively different representations or dedicate specific heads to tool routing. The Tigges et al. [2024] finding that circuit structures are consistent across model scale provides *some* justification for the GPT-2-SMALL results, but direct probing of larger instruction-tuned models is essential future work.

Probing detects correlation, not causality. Linear probes measure the decodability of information from representations, but high probe accuracy does not prove that the probed information is causally used for the behavior [Sharkey et al., 2025]. The perturbation experiments provide complementary causal evidence at the behavioral level, but activation patching on specific attention heads—the method used in circuit-level ACDC analysis [Conmy et al., 2023]—would be needed to establish causal responsibility at the circuit level.

No circuit-level analysis. This work identifies *where* information is encoded but not *which* circuits compute it. ACDC circuit discovery [Conmy et al., 2023] was not applied; applying it to GPT-2-SMALL tool-selection prompts would require GPU resources and would identify the specific attention heads performing cross-tool comparison in layers 7–9.

Fixed tool descriptions. Our curated descriptions are drawn from real API documentation but are fixed in length and style. Real-world tool registries vary enormously in description quality. Systems with short, vague, or redundant descriptions would likely show lower semantic accuracy and higher position bias than reported here.

5.3 Broader Implications

These findings have direct engineering implications. First, within-category tool selection is the hardest case (37.5% semantic accuracy)—yet it is the most common scenario in multi-tool agents that register many tools with overlapping functions. Practitioners should prioritize making within-category descriptions maximally distinctive. Second, tool names carry independent priors: agents deploying tools with highly recognizable names (e.g., Google-branded APIs) may find that description content is partially overridden. Third, the layer 9 peak in position probing suggests that interpretability interventions—such as activation steering—applied at that layer could potentially redirect tool selection without modifying the model's weights, an exciting direction for robust tool-routing systems.

6 Conclusion

We presented the first empirical mechanistic characterization of tool selection in large language models, combining behavioral analysis, semantic probing, causal perturbation, and layer-wise representational probing. Our four experiments converge on a coherent picture.

At the behavioral level, tool selection is driven primarily by semantic description content: cosine similarity predicts 40% of selections (vs. 25% chance), selection stability across orderings is 90%, and adversaially swapping descriptions changes selection in 67% of cases. Positional bias exists in uncontrolled settings but is not significant in controlled rotation experiments, indicating that high

position-0 rates observed in prior work partly reflect correlated description quality. Tool names carry an independent prior that can override description content for highly distinctive names.

At the mechanistic level, we uncover a clear dissociation within the GPT-2-SMALL transformer: tool category is fully encoded from layer 1 (100% probe accuracy), consistent with lexical encoding in token embeddings. In contrast, the identification of which specific tool position is semantically correct emerges progressively, rising from 35% at layer 1 to 68% at layer 9, before declining. This gradient establishes that semantic matching is a non-trivial multi-layer computation concentrated in the middle-to-late transformer layers.

Future work. The most important next step is applying ACDC circuit discovery [Conmy et al., 2023] to identify the specific attention heads responsible for cross-tool comparison in layers 7–9 of GPT-2-small. Extending layer-wise probing to instruction-tuned models (LLaMA-3.1-8B-Instruct, GPT-4.1-mini) would test whether tool-use fine-tuning creates specialized selection circuits or reuses the same mid-layer comparison mechanism. Sparse autoencoder (SAE) analysis [Sharkey et al., 2025] could identify monosemantic features that activate for tool-relevant concepts, enabling targeted activation steering to correct selection errors without weight updates. We release all datasets, code, and experimental results to support replication and extension of this work.

References

Adam Blankenstein, Sherry Yu, Zhenghao Li, Vassilis Plachouras, Bidisha Sengupta, Philip Torr, Yarin Gal, Amon Paren, and Adel Bibi. BiasBusters: Uncovering and mitigating tool selection bias in large language models. *arXiv preprint arXiv:2510.00307*, 2025.

Arthur Conmy, Augustine N. Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adrià Garriga-Alonso. Towards automated circuit discovery for mechanistic interpretability. In *Advances in Neural Information Processing Systems*, volume 36, 2023. arXiv:2304.14997.

Kait Healy, Bharathi Srinivasan, Visakh Madathil, and Jing Wu. Internal representations as indicators of hallucinations in agent tool selection. In *AAAI Workshop on Neuro-Symbolic and Reliable AI*, 2026. arXiv:2601.05214.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

Jack Merullo, Carsten Eickhoff, and Ellie Pavlick. Circuit component reuse across tasks in transformer language models. In *International Conference on Learning Representations*, 2024. arXiv:2310.08744.

Yujia Qin, Shihao Liang, Yining Ye, Kunliang Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations*, 2024. arXiv:2307.16789.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023. arXiv:2302.04761.

Lee Sharkey, Bilal Chughtai, Jack Lindsey, et al. Open problems in mechanistic interpretability. *Transactions on Machine Learning Research*, 2025. arXiv:2501.16496.

Curt Tigges, Michael Hanna, Qinan Yu, and Stella Biderman. LLM circuit analyses are consistent across training and scale. In *Advances in Neural Information Processing Systems*, volume 37, 2024. arXiv:2407.10827.

Shishir G. Yan et al. Berkeley function calling leaderboard. *Online Leaderboard*, 2023. `https://gorilla.cs.berkeley.edu/leaderboard.html`.

A Appendix

A.1 Dataset Examples

Same-category example (fine-grained semantic matching):

> **Query**: "What's the forecast for Los Angeles this week?"
> **Tools**: `[openweathermap_api, weatherapi_com, world_weather_online, weather_gov_api]`
> **LLM selection**: `openweathermap_api` (position 0)

Mixed-category example (coarse routing):

> **Query**: "Compute the integral of x^3 from 0 to 5."
> **Tools**: `[wolfram_alpha_api, openweathermap_api, google_search_api, jupyter_kernel_api]`
> **LLM selection**: `wolfram_alpha_api` (position 0)

Adversarial perturbation example:

> **Query**: "What's the forecast for London this weekend?"
> **Condition**: Adversarial — `wolfram_alpha_api` carries the weather description
> **LLM selection**: `wolfram_alpha_api` (model follows the description, not the name)

A.2 Implementation Details

Table 7 summarizes all key hyperparameters used across experiments.

Table 7: Hyperparameters used across all experiments.

Parameter	Value	Rationale
LLM temperature	0.0	Deterministic selections
LLM max tokens	50	Tool name only
Runs per example	3	Consistency measurement
Probe regularization C	1.0	Default; balanced
Probe max iterations	2000	Convergence
Cross-validation folds	5	Standard; stratified
GPT-2 max context	512	Fits full prompt
Random seed	42	Reproducibility

A.3 Software and Hardware

All experiments ran on a single CPU (no GPU). LLM experiments took approximately 15 minutes; probing experiments took approximately 8 minutes. Key library versions: sentence-transformers (latest), transformers 5.3.0, scikit-learn 1.8.0, scipy 1.17.1, openai 2.26.0.

A.4 Last-token Probe Accuracy by Layer Group

Table 8: Correct tool position probe accuracy using the last-token representation (GPT-2-SMALL, 5-fold CV). Last-token accuracy is high from layer 1, supporting the hypothesis that the last-token position aggregates selection-relevant context from the beginning of processing. Chance is 0.25.

Layer Group	**Mean Accuracy**	**Chance**
Early (1–4)	0.723	0.25
Middle (5–8)	0.733	0.25
Late (9–12)	0.730	0.25
Best (layer 8/10)	**0.810**	0.25